

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par Amina Abid

Année Universitaire : 2025-2026

Table des matières

1	Introduction et Objectifs du Projet	3
1.1	Objectifs du Projet	3
1.2	Formalisation du Problème	3
2	Exploration des données (EDA)	3
2.1	Vue Générale	3
2.2	Distribution de la variable cible	3
2.3	Distribution des variables numériques	4
2.4	Analyse RFM	4
2.5	Analyse des variables catégorielles	5
2.6	Corrélation entre variables	5
2.7	Résumé des problèmes identifiés	6
3	Pipeline de Préprocessing — preprocessing.py	6
3.1	Nettoyage des données et suppression du Data Leakage	7
3.1.1	Variables non prédictives ou sans information	7
3.1.2	Variables redondantes (multicolinéarité parfaite)	7
3.1.3	Variables supprimées pour Data Leakage	7
3.1.4	Variables manquantes ou aberrantes	8
3.1.5	Synthèse du nettoyage	8
3.1.6	Conséquence du Data Leakage	8
3.2	Parsing des Dates	8
3.3	Feature Engineering	9
3.4	Séparation du jeu de données (Train/Test Split)	9
3.4.1	Stratification du split	9
3.4.2	Résultats du split	10
3.4.3	Paramètres de reproductibilité	10
3.5	Imputation des valeurs manquantes	10
3.6	Encodage des variables catégorielles	10
3.6.1	Encodage ordinal	11
3.6.2	Encodage par target encoding	11
3.6.3	Encodage one-hot	11
3.6.4	Synthèse	11
3.7	Suppression de la multicolinéarité	11
3.8	Normalisation des variables numériques — RobustScaler	12
3.8.1	Principe du RobustScaler	12
3.8.2	Application dans le pipeline	12
3.8.3	Implémentation	12
3.8.4	Conclusion	13
3.9	Rééquilibrage — SMOTE	13
4	Modélisation — train_model.py	13
4.1	Séparation Train/Test Stratifiée	13
4.2	Métriques d'évaluation utilisées	13
4.3	Résultats des modèles supervisés	13
4.3.1	1. Régression Logistique	14
4.3.2	2. Random Forest	14
4.3.3	3. XGBoost	14
4.3.4	Stratégie commune d'entraînement	15
4.4	Tableau comparatif des modèles	15
4.5	Conclusion de la phase d'entraînement	16

5	Optimisation des hyperparamètres	16
5.0.1	GridSearchCV	16
5.0.2	Optuna (Optimisation bayésienne)	16
5.1	Comparaison des méthodes d'optimisation	17
5.2	Analyse de l'importance des variables	17
5.3	Analyse de l'overfitting du modèle	18
5.3.1	Interprétation des résultats	18
5.4	Conclusion	19
6	Déploiement — Application Web Flask	20
6.1	Architecture des endpoints	20
6.2	Pipeline de prétraitement en production	20
6.3	Features les plus importantes du modèle	20
6.4	Exemple d'utilisation de l'API	21
6.5	Règle de décision (Risk Engine)	21
6.6	Tests manuels des prédictions	21
6.6.1	Client VIP fidèle	21
6.6.2	Client à risque de churn	22
6.6.3	Client moyen occasionnel	22
6.6.4	Conclusion des tests manuels	22
6.7	Conclusion du déploiement	22
7	Structure du Projet	23
8	Conclusion	23

1 Introduction et Objectifs du Projet

Ce projet s'inscrit dans le cadre de l'atelier pratique de Machine Learning appliqué à l'analyse comportementale d'une clientèle d'e-commerce. L'objectif principal est de construire un système de prédiction du **churn client** (départ d'un client) à partir de ses données transactionnelles et comportementales.

Le churn est l'un des défis stratégiques majeurs du commerce en ligne : acquérir un nouveau client coûte en moyenne **5 à 7 fois plus cher** que de fidéliser un client existant. Identifier les clients à risque avant qu'ils ne partent permet de mettre en place des actions de rétention ciblées et rentables.

1.1 Objectifs du Projet

1. Explorer et préparer un dataset réel de 4372 clients (52 features)
2. Implémenter un pipeline de préprocessing complet en 9 étapes
3. Comparer 3 modèles supervisés + 1 analyse KMeans non supervisée
4. Optimiser le meilleur modèle avec GridSearchCV puis Optuna
5. Déployer le modèle via une API Flask avec interface web

1.2 Formalisation du Problème

Il s'agit d'un problème de **classification binaire supervisée** :

- **Variable cible** : $\text{Churn} \in \{0, 1\}$ — 0 = client fidèle, 1 = client parti
- **Features** : 51 variables décrivant le comportement d'achat, le profil démographique et l'activité du compte
- **Déséquilibre des classes** : 66,7% de non-churners (0) vs 33,3% de churners (1) → nécessite une gestion spécifique

2 Exploration des données (EDA)

2.1 Vue Générale

Statistiques clés du dataset				
4 372	52	1 454	33,3 %	37
Clients	Features	Churners	Taux Churn	Pays

Le dataset `retail_customers_COMPLETE_CATEGORICAL.csv` contient 4372 enregistrements et 52 colonnes décrivant le comportement d'achat de clients d'un e-commerce de cadeaux basé principalement au Royaume-Uni (90,4% des clients).

2.2 Distribution de la variable cible

La distribution montre un déséquilibre modéré entre les classes, avec environ un tiers de clients churners.

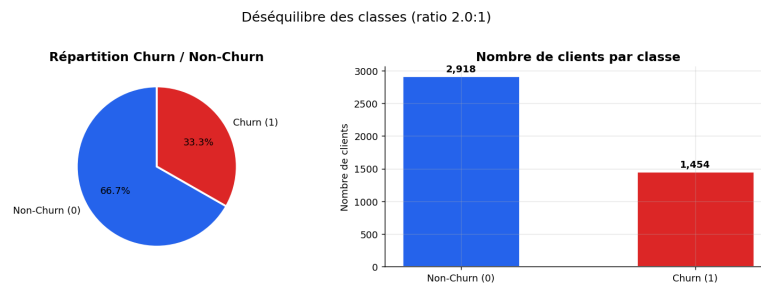


FIGURE 1 – Distribution de la variable cible (Churn vs Non-Churn)

2.3 Distribution des variables numériques

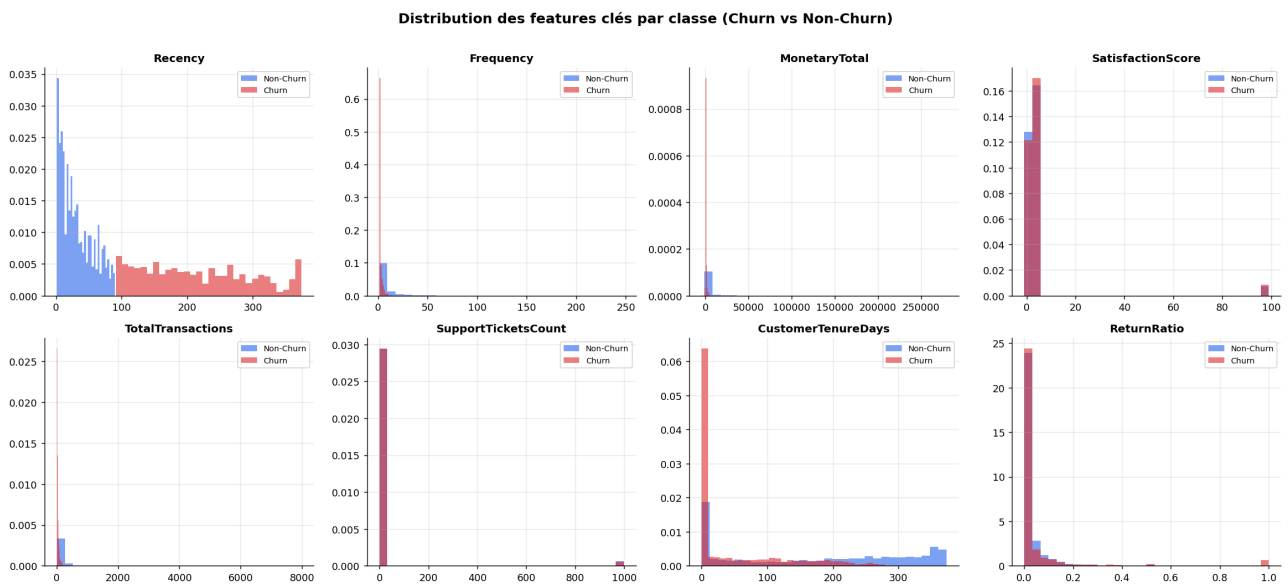


FIGURE 2 – Distribution des principales variables numériques (RFM et comportement client)

On observe des distributions fortement asymétriques sur plusieurs variables monétaires, justifiant l'utilisation d'un RobustScaler.

2.4 Analyse RFM

Les boxplots montrent clairement que les clients churners présentent une récence plus élevée et une fréquence d'achat plus faible.

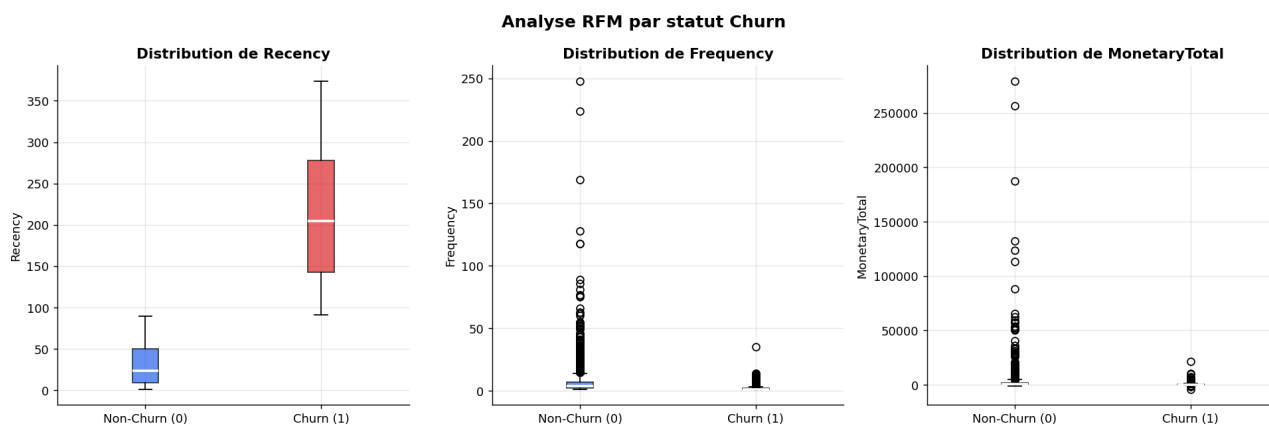


FIGURE 3 – Analyse RFM (Recency, Frequency, Monetary) selon le churn

2.5 Analyse des variables catégorielles

Certaines catégories présentent des différences significatives de taux de churn, notamment les segments clients et les niveaux de fidélité.

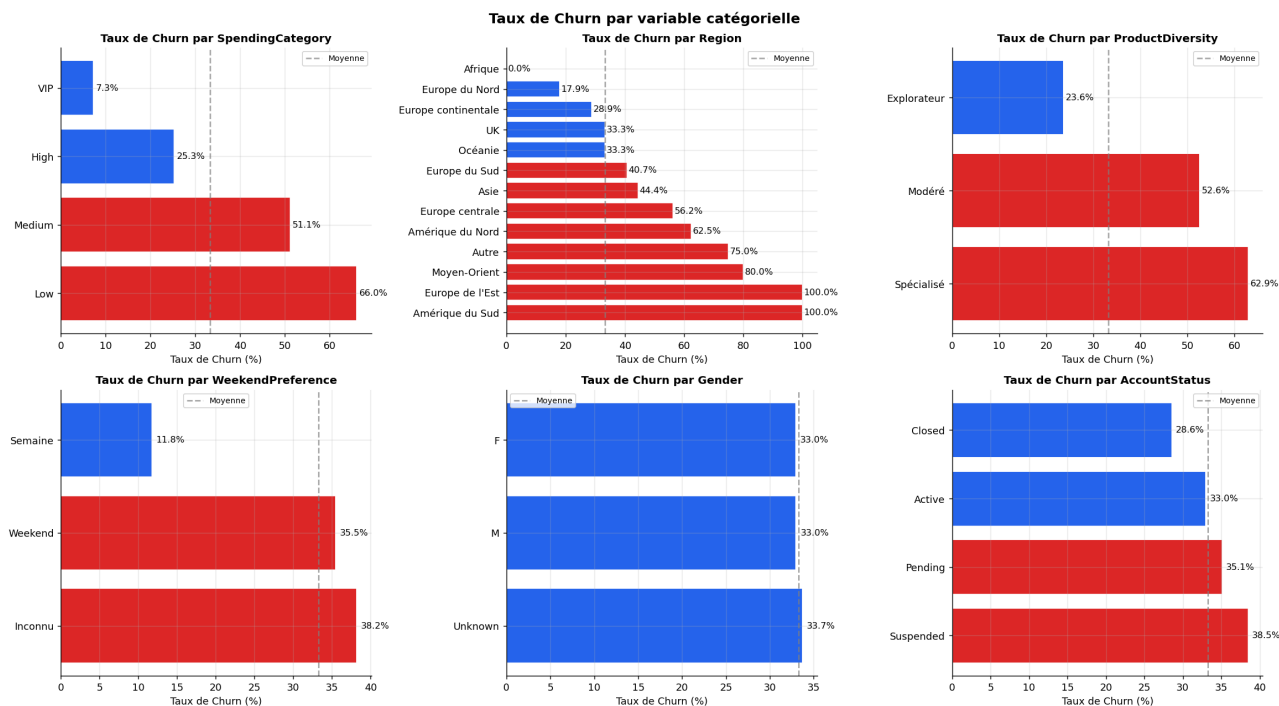


FIGURE 4 – Relation entre variables catégorielles et churn

2.6 Corrélation entre variables

La matrice met en évidence des corrélations fortes entre certaines variables (ex : MonetaryTotal et TotalQuantity), justifiant la suppression de la multicollinéarité.

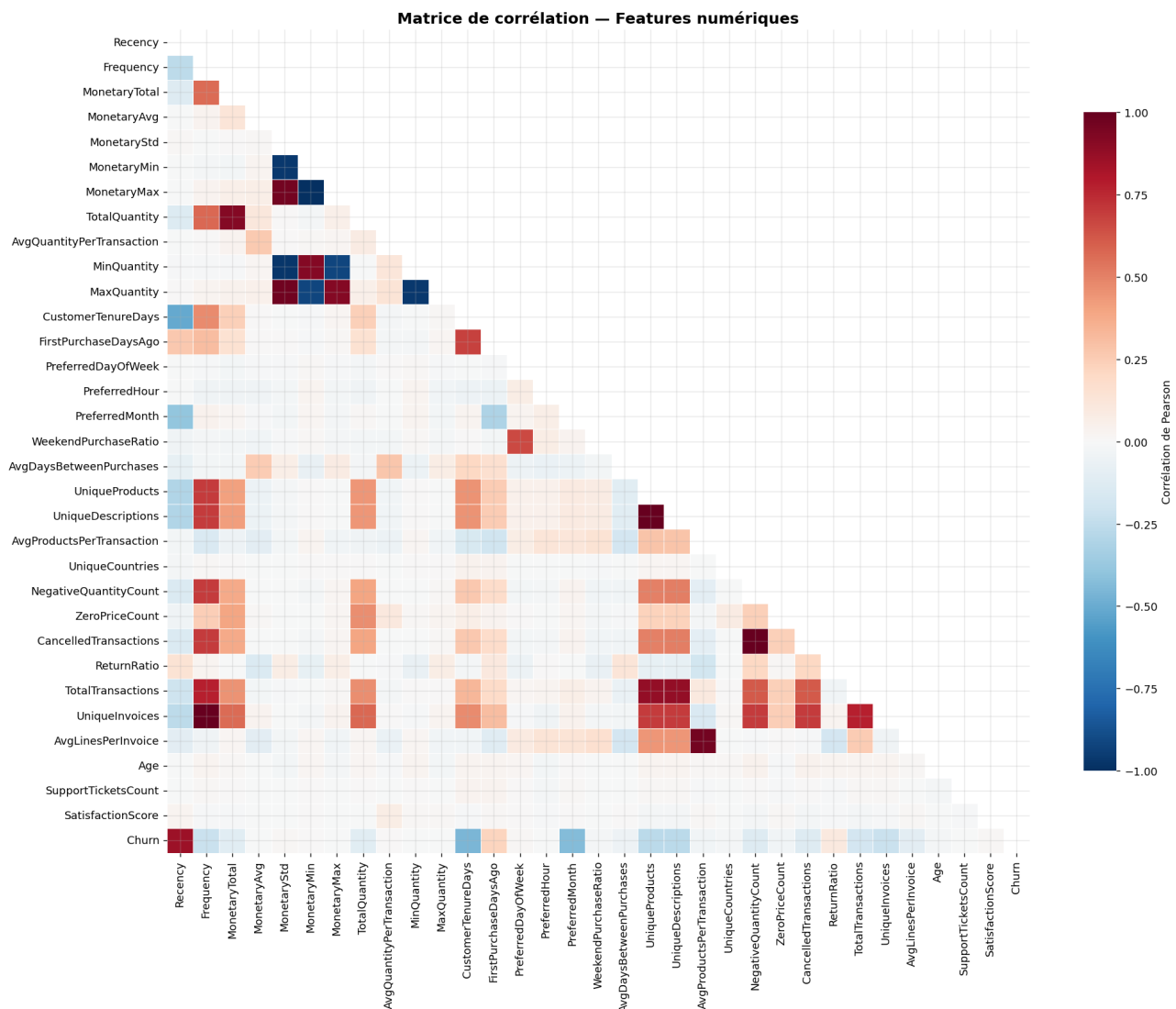


FIGURE 5 – Matrice de corrélation des variables numériques

2.7 Résumé des problèmes identifiés

Problèmes détectés et corrections appliquées

Data Leakage (critique)	Variables supprimées : ChurnRiskCategory, CustomerType, RFMSegment, etc.
Valeurs manquantes	AvgDaysBetweenPurchases (1,8%) imputée par médiane.
Outliers	SatisfactionScore et SupportTicketsCount corrigés (clipping).
Variance nulle	NewsletterSubscribed supprimée.
Redondance	Variables fortement corrélées supprimées après analyse.

3 Pipeline de Préprocessing — preprocessing.py

Le fichier `preprocessing.py` implémente un pipeline complet en **8 étapes** transformant le dataset brut (4372×52) en données prêtes pour les modèles ML. La règle d'or : les transformations sont apprises sur `X_train` uniquement (fit), puis appliquées sur `X_test` (transform).

N°	Nom	Fonction	Description
1	Nettoyage	<code>clean_data()</code>	18 colonnes supprimées (leakage, redondances, variance nulle), valeurs aberrantes corrigées
2	Parsing dates	<code>parse_dates()</code>	<code>RegistrationDate</code> → <code>RegMonth</code> , <code>RegWeekday</code> , <code>RegIsWeekend</code> , <code>AccountAgeDays</code> , <code>IsNewClient</code> , <code>RegSeason</code>
3	Feature Engineering	<code>feature_engineering()</code>	5 nouvelles features comportementales
0	Train/Test Split	<code>train_test_split()</code>	80/20 stratifié — avant toute transformation apprise
4	Imputation	<code>impute_missing()</code>	Médiane pour numériques, mode pour catégorielles
5	Encodage	<code>encode_categoricals()</code>	OrdinalEncoder + TargetEncoder (Region) + One-Hot Encoding
6	Multicolinéarité	<code>remove_multicollinearity()</code>	Suppression si $ r > 0.85$ (v3) — 10 colonnes supprimées
7	Normalisation	<code>scale_features()</code>	RobustScaler sur 25 colonnes continues (v3)
8	Rééquilibrage	<code>balance_classes()</code>	SMOTE : $\{0 : 2334, 1 : 1163\} \rightarrow \{0 : 2334, 1 : 2334\}$

3.1 Nettoyage des données et suppression du Data Leakage

Une étape critique du prétraitement consiste à identifier et supprimer les variables susceptibles d'introduire un **data leakage**, ainsi que celles non pertinentes ou redondantes. Cette étape garantit que le modèle apprend uniquement à partir d'informations disponibles en situation réelle de prédiction.

3.1.1 Variables non prédictives ou sans information

Certaines variables ont été supprimées car elles n'apportent aucune information utile au modèle :

- `CustomerID` : identifiant unique propre à chaque client, sans pouvoir prédictif.
- `NewsletterSubscribed` : variable constante (tous les clients ont la valeur "Yes"), donc variance nulle.
- `LastLoginIP` : donnée brute remplacée par des variables dérivées plus pertinentes (ex : type d'IP).

3.1.2 Variables redondantes (multicolinéarité parfaite)

Certaines colonnes ont été supprimées car elles sont strictement équivalentes à d'autres :

- `UniqueInvoices` = `Frequency` (corrélation = 1.00)
- `CancelledTransactions` = `NegativeQuantityCount` (corrélation = 1.00)

D'autres variables présentent une forte multicolinéarité ($|\rho| > 0.9$), notamment dans les variables monétaires et de quantités (ex : `MonetaryMin`, `MonetaryMax`, `MonetaryStd`). Bien que certaines aient été conservées pour préserver l'information globale, une vigilance particulière a été appliquée pour éviter la redondance excessive.

3.1.3 Variables supprimées pour Data Leakage

Plusieurs variables ont été identifiées comme introduisant un **data leakage direct**, car elles encodent explicitement ou implicitement la variable cible (**Churn**) :

- `ChurnRiskCategory` : la modalité "Critique" correspond à 100% de churn, ce qui revient à révéler directement la cible.
- `CustomerType` : la catégorie "Perdu" correspond systématiquement à des clients churnés.
- `Recency` : fortement corrélée avec le churn ($\rho = 0.86$), ce qui peut indiquer une dépendance directe à la définition du churn.

- **FavoriteSeason** : forte corrélation avec le churn (ex : Automne $\approx$ 98% de non-churn), ce qui indique une fuite d'information.
- **RFMSegment**, **LoyaltyLevel** : variables dérivées de comportements clients (récence, fréquence, montant), potentiellement calculées avec des informations postérieures à l'événement de churn.
- **FirstPurchaseDaysAgo**, **CustomerTenureDays**, **Age**, **UniqueCountries** : variables redondantes avec **AccountAgeDays** ou contenant une information similaire.
- **Age** et **AgeCategory** : ces variables ont été supprimées en raison d'un **taux élevé de valeurs manquantes (30%)**. Une telle proportion rend les méthodes d'imputation peu fiables, car elles risquent d'introduire un biais significatif et de dégrader la qualité des données. De plus, **AgeCategory** étant une variable dérivée de **Age**, elle hérite des mêmes limitations, ce qui justifie leur suppression conjointe.

3.1.4 Variables manquantes ou aberrantes

Plusieurs corrections ont été appliquées afin d'améliorer la qualité des données

- **SupportTicketsCount** : les valeurs aberrantes négatives (43 occurrences) ont été remplacées par 0.
- **SatisfactionScore** : les valeurs incohérentes supérieures à 10 (114 occurrences) ont été tronquées à 10 afin de respecter le domaine de définition.
- **AvgDaysBetweenPurchases** : 79 valeurs invalides ont été converties en valeurs manquantes (NaN) pour être traitées ultérieurement par imputation.

3.1.5 Synthèse du nettoyage

Au total, **18 colonnes ont été supprimées** lors de cette étape de prétraitement, réduisant le jeu de données de **52 à 34 variables**.

Cette réduction s'explique par l'élimination des variables non prédictives, redondantes ou introduisant un *data leakage*, permettant ainsi d'obtenir un jeu de données plus cohérent, pertinent et adapté à l'apprentissage de modèles généralisables.

3.1.6 Conséquence du Data Leakage

Impact du Data Leakage

Avant suppression : les modèles atteignent des performances artificiellement parfaites ($F1 \approx 1.0$, $AUC-ROC \approx 1.0$), révélant une fuite d'information.

Après nettoyage (version v3) : les performances deviennent réalistes (XGBoost : $F1 = 0.739$, $AUC-ROC = 0.895$), reflétant une capacité de généralisation réelle.

Principe fondamental : une variable ne doit être utilisée que si elle est disponible **avant** la survenue du churn dans un contexte réel.

3.2 Parsing des Dates

La colonne **RegistrationDate** existe en plusieurs formats inconsistants (ISO, DD/MM/YYYY, MM/DD/YYYY, variantes à 2 chiffres). La fonction `parse_dates()` unifie ces formats avec détection par règle explicite et extrait 6 nouvelles features :

- **RegMonth** : Mois d'inscription (1–12)
- **RegWeekday** : Jour de la semaine (0=Lundi)
- **RegIsWeekend** : Binaire : inscription un week-end
- **AccountAgeDays** : Ancienneté du compte en jours
- **IsNewClient** : Binaire : compte < 1 an
- **RegSeason** : Saison d'inscription

3.3 Feature Engineering

Afin d'enrichir la représentation des données, plusieurs variables comportementales ont été créées à partir des features existantes. Chaque feature vise à capturer un aspect spécifique du comportement client.

— AvgBasketValue

$$\text{AvgBasketValue} = \frac{\text{MonetaryTotal}}{\text{Frequency} + 1}$$

Représente la valeur moyenne dépensée par transaction. Elle permet d'identifier les clients à forte valeur même avec une faible fréquence d'achat.

— CancellationRate

$$\text{CancellationRate} = \frac{\text{NegativeQuantityCount}}{\text{TotalTransactions} + 1}$$

Mesure le taux d'annulation des transactions. Un taux élevé peut refléter une insatisfaction ou un comportement instable.

— SpendingVolatility

$$\text{SpendingVolatility} = \frac{\text{MonetaryStd}}{|\text{MonetaryAvg}| + 1}$$

Quantifie l'instabilité des dépenses. Une forte volatilité indique un comportement d'achat irrégulier.

— EngagementScore

$$\text{EngagementScore} = \frac{\text{SatisfactionScore}}{\text{SupportTicketsCount} + 1}$$

Combine satisfaction et réclamations clients. Un score faible indique un engagement client faible et un risque accru de churn.

— ProductsPerTransaction

$$\text{ProductsPerTransaction} = \frac{\text{UniqueProducts}}{\text{TotalTransactions} + 1}$$

Mesure la diversité des produits achetés par transaction. Permet d'identifier les clients explorateurs vs. répétitifs.

3.4 Séparation du jeu de données (Train/Test Split)

Après l'étape de feature engineering, le jeu de données est séparé en variables explicatives (X) et variable cible (y) :

$$X = \text{features}, \quad y = \text{Churn}$$

La distribution de la variable cible est d'abord vérifiée afin d'identifier un éventuel déséquilibre de classes.

3.4.1 Stratification du split

Afin de garantir que les proportions de churn et non-churn soient conservées dans les deux ensembles, un *split stratifié* est appliqué :

$$\text{Train/Test} = 80\%/20\%$$

— **Train set** : utilisé pour l'apprentissage du modèle

— **Test set** : utilisé uniquement pour l'évaluation finale

Le split est réalisé avec un paramètre `stratify = y`, ce qui permet de préserver la distribution de la cible dans les deux ensembles et d'éviter un biais d'échantillonnage.

3.4.2 Résultats du split

$$X_{train} = (3497, n), \quad X_{test} = (875, n)$$

où n représente le nombre de features après feature engineering et encodage.

3.4.3 Paramètres de reproductibilité

Le paramètre `random_state` est fixé afin de garantir la reproductibilité des résultats lors des différentes exécutions du pipeline.

3.5 Imputation des valeurs manquantes

L'imputation des valeurs manquantes a été réalisée en fonction de la nature des variables et de leur distribution. L'objectif est de conserver l'intégrité statistique des données tout en évitant l'introduction de biais.

- **AvgDaysBetweenPurchases** : 79 valeurs manquantes (1,8%)

Imputation = médiane

La médiane a été choisie car elle est robuste aux valeurs extrêmes et permet de préserver la distribution initiale des données.

- **Variables numériques générales**

Imputation = médiane

Toutes les variables numériques présentant des valeurs manquantes sont imputées par leur médiane afin de limiter l'influence des outliers.

- **Variables catégorielles**

Imputation = mode (valeur la plus fréquente)

Les variables catégorielles sont complétées par leur modalité la plus fréquente, ce qui permet de conserver la structure dominante des données.

- **Variables critiques (SatisfactionScore, AvgDaysBetweenPurchases)**

Imputation = KNN Imputer ($k = 5$)

Pour les variables jugées sensibles et potentiellement corrélées avec d'autres features, une imputation par K-Nearest Neighbors a été utilisée afin de mieux préserver les relations locales entre observations.

Règle méthodologique importante : Le `fit` des imputers est effectué uniquement sur `X_train`, puis le `transform` est appliqué sur `X_train` et `X_test` séparément, afin d'éviter toute fuite de données (*data leakage*).

Résultat final : aucune valeur manquante restante après imputation.

3.6 Encodage des variables catégorielles

Les variables catégorielles ont été encodées selon leur nature (ordinaire ou nominale) afin de les transformer en représentations numériques exploitables par les modèles de machine learning, tout en conservant au maximum l'information sémantique.

3.6.1 Encodage ordinal

Les variables présentant un ordre logique intrinsèque ont été encodées via un *Ordinal Encoding* :

- `SpendingCategory` : $Low < Medium < High < VIP \rightarrow$ encodée en (0, 1, 2, 3)
- `BasketSizeCategory` : $Petit < Moyen < Grand$

Cet encodage est réalisé à l'aide de `OrdinalEncoder`, avec la gestion des catégories inconnues via `unknown_value = -1`. Cette approche permet de préserver la relation d'ordre entre les modalités, contrairement à un encodage one-hot qui la détruirait.

3.6.2 Encodage par target encoding

La variable `Region` a été encodée à l'aide du *Target Encoding*, où chaque modalité est remplacée par la moyenne de la variable cible (`Churn`) correspondante.

$$Region_{enc} = \mathbb{E}[Churn \mid Region]$$

Un paramètre de lissage (`smoothing = 1.0`) a été utilisé afin de réduire le surapprentissage sur les régions peu représentées.

Important : l'encodage est ajusté uniquement sur `X_train` et `y_train`, puis appliqué à `X_test` afin d'éviter toute fuite de données (*data leakage*).

3.6.3 Encodage one-hot

Les variables nominales sans ordre logique ont été transformées via un encodage one-hot :

- `PreferredTimeOfDay`
- `WeekendPreference`
- `ProductDiversity`
- `Gender`
- `AccountStatus`
- `RegSeason`

Chaque modalité est transformée en variable binaire (0/1) à l'aide de `pd.get_dummies()`. Les colonnes du train et du test sont ensuite alignées afin de garantir une structure identique entre les deux jeux de données (valeurs manquantes remplies par 0).

3.6.4 Synthèse

Après encodage, le jeu de données passe de **43 variables initiales à 58 variables finales**, après expansion des variables catégorielles par one-hot encoding.

$$X_{train} = (3497, 58), \quad X_{test} = (875, 58)$$

Cette étape permet de rendre toutes les variables compatibles avec les algorithmes de machine learning tout en conservant leur information structurelle.

3.7 Suppression de la multicollinéarité

Afin de réduire la redondance entre les variables et d'améliorer la robustesse des modèles, une analyse de corrélation a été réalisée avec un seuil fixé à $|r| > 0.85$.

Lorsqu'une forte corrélation est détectée, une seule variable est conservée selon sa pertinence métier et sa stabilité statistique.

Paire de variables	$ r $	Variable supprimée	Justification
UniqueProducts ↔ UniqueDescriptions	1.000	UniqueProducts	Redondance parfaite (variables identiques)
MonetaryMin ↔ MonetaryMax	0.995	MonetaryMin	Forte redondance sur les statistiques de dépenses
MinQuantity ↔ MaxQuantity	0.989	MinQuantity	Redondance sur les extrêmes de quantité
MonetaryStd ↔ MaxQuantity	0.981	MonetaryStd	Corrélation structurelle élevée
ReturnRatio ↔ CancellationRate	0.966	CancellationRate	Mesure similaire du comportement d'annulation
AvgProductsPerTransactic ↔ AvgLinesPerInvoice	0.963	AvgProductsPerTransactic	Redondance sur la densité d'achat
WeekendPurchaseRatio ↔ WeekendPreference	0.913	WeekendPurchaseRatio	Feature dérivée redondante avec variable catégorielle
MonetaryTotal ↔ TotalQuantity	0.900	TotalQuantity	Forte corrélation volume/valeur d'achat
UniqueDescriptions ↔ TotalTransactions	0.878	UniqueDescriptions	Redondance partielle sur la diversité d'achat

Résultat final : 10 variables supprimées → **48 features finales**.

Nombre final de features = 48

3.8 Normalisation des variables numériques — RobustScaler

La normalisation des variables numériques est une étape essentielle afin d'assurer une convergence stable des algorithmes sensibles aux échelles (notamment les modèles linéaires et les réseaux neuronaux).

Dans cette version du pipeline, le **RobustScaler** a été utilisé à la place du **StandardScaler**, en raison de sa robustesse face aux valeurs extrêmes.

3.8.1 Principe du RobustScaler

Le **RobustScaler** normalise les données en utilisant la médiane et l'intervalle interquartile (IQR), ce qui le rend moins sensible aux outliers que les méthodes basées sur la moyenne.

$$X_{scaled} = \frac{X - \text{médiane}}{IQR}$$

Cette propriété est particulièrement adaptée à notre dataset contenant des valeurs extrêmes sur les variables monétaires et comportementales.

3.8.2 Application dans le pipeline

Seules les variables numériques continues ont été normalisées. Les variables binaires issues de l'encodage one-hot (0/1) ont été exclues afin de préserver leur interprétation et éviter toute distorsion.

- **Variables normalisées :** 25 colonnes numériques continues
- **Variables exclues :** 23 colonnes binaires (One-Hot Encoding)

3.8.3 Implémentation

L'ajustement (**fit**) du scaler est effectué uniquement sur **X_train**, puis appliqué à **X_test** afin d'éviter toute fuite de données (*data leakage*).

3.8.4 Conclusion

Cette normalisation permet d'améliorer la stabilité des modèles tout en conservant la structure des variables binaires, assurant ainsi un prétraitement cohérent et robuste.

3.9 Rééquilibrage — SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) génère des exemples synthétiques de la classe minoritaire (churners) en interpolant entre des exemples voisins ($k=5$).

Ensemble	Avant SMOTE	Après SMOTE	Ratio
X_train (80 %)	{0 :2334, 1 :1163}	{0 :2334, 1 :2334}	1 :1
X_test (20 %)	{0 :584, 1 :291}	<i>inchangé</i>	2 :1

Note critique sur le SMOTE

Le SMOTE est appliqué sur l'ensemble du **X_train** avant la validation croisée dans Optuna → le CV score est légèrement surestimé (gap CV-Test de 0,12). Solution recommandée : **imblearn.pipeline.Pipeline** pour confiner le SMOTE dans chaque fold d'entraînement unique-ment.

4 Modélisation — train_model.py

4.1 Séparation Train/Test Stratifiée

Le dataset est divisé en 80% entraînement et 20% test avec stratification pour conserver la distribution des classes :

Ensemble	Taille	Non-Churn (0)	Churn (1)
X_train (80%)	4 668	2 334 (50%)	2 334 (50%)
X_test (20%)	875	584 (66.7%)	291 (33.3%)

4.2 Métriques d'évaluation utilisées

Dans ce problème de prédiction du churn, les données sont équilibrées après application de SMOTE. Cependant, la détection des clients susceptibles de partir reste prioritaire. Ainsi, les métriques suivantes ont été retenues :

Métriques d'évaluation

- **Recall** : mesure la capacité du modèle à détecter les clients churners (priorité métier).
- **Precision** : mesure la fiabilité des prédictions positives.
- **F1-score** : compromis entre precision et recall.
- **AUC-ROC** : capacité globale du modèle à discriminer les classes.

Le Recall est particulièrement important car un churn non détecté représente une perte directe pour l'entreprise.

4.3 Résultats des modèles supervisés

Trois modèles de classification ont été entraînés et évalués sur le même jeu de données :

4.3.1 1. Régression Logistique

La régression logistique est un modèle linéaire probabiliste basé sur la fonction sigmoïde. Elle estime la probabilité de churn en combinant linéairement les variables explicatives.

Logique de fonctionnement :

- Modèle linéaire sur les features
- Application d'une fonction sigmoïde pour obtenir une probabilité
- Classification basée sur un seuil (0.5 ou ajusté)

Paramètres utilisés :

- `max_iter` = 2000 (convergence assurée)
- `solver` = `lbfgs`
- `C` = 1.0 (régularisation L2)
- `class_weight` = `balanced` (gestion du déséquilibre des classes)
- `random_state` = 42

Objectif : Servir de modèle baseline interprétable pour comparer les performances des modèles non linéaires.

4.3.2 2. Random Forest

Random Forest est un modèle d'ensemble basé sur le bagging d'arbres de décision.

Logique de fonctionnement :

- Construction de plusieurs arbres de décision indépendants
- Agrégation des prédictions par vote majoritaire
- Réduction de la variance par échantillonnage aléatoire

Paramètres utilisés :

- `n_estimators` = 100
- `max_depth` = `None` (arbres complets par défaut)
- `min_samples_split` = 2
- `min_samples_leaf` = 1
- `max_features` = `sqrt` / `log2` (sélection aléatoire des features)
- `class_weight` = `balanced`
- `n_jobs` = -1 (parallélisation)
- `random_state` = 42

Objectif : Capturer les relations non linéaires tout en réduisant le surapprentissage.

4.3.3 3. XGBoost

XGBoost est un algorithme de gradient boosting qui construit les arbres de manière séquentielle, chaque nouvel arbre corrigeant les erreurs des précédents.

Logique de fonctionnement :

- Apprentissage séquentiel (boosting)
- Minimisation d'une fonction de perte via gradient
- Correction progressive des erreurs du modèle

Gestion du déséquilibre :

- `scale_pos_weight` calculé dynamiquement :

$$\text{scale_pos_weight} = \frac{n_{\text{negatifs}}}{n_{\text{positifs}}}$$

Paramètres de base utilisés :

- `n_estimators` = 100
- `max_depth` = 3 à 7 (selon optimisation)
- `learning_rate` = 0.05 à 0.2
- `subsample` = 0.8 à 1.0
- `colsample_bytree` = 0.5 à 1.0
- `min_child_weight` = 1 à 15
- `reg_lambda`, `reg_alpha` (régularisation L2 et L1)
- `gamma` (contrôle de complexité des splits)
- `eval_metric` = logloss
- `random_state` = 42

Objectif : Maximiser la performance prédictive sur des données tabulaires complexes avec forte capacité de généralisation.

4.3.4 Stratégie commune d'entraînement

Pour les trois modèles, la stratégie suivante a été appliquée :

- Validation croisée stratifiée (5-fold)
- Gestion du déséquilibre des classes (`class_weight` / `scale_pos_weight`)
- Évaluation sur plusieurs métriques (Recall, F1, AUC-ROC)
- Séparation stricte train/test (80/20)

4.4 Tableau comparatif des modèles

Modèle	Accuracy	Precision	Recall	F1-score	AUC-ROC
Régression Logistique	0.7029	0.5320	0.8866	0.6649	0.8453
Random Forest	0.7269	0.5558	0.8900	0.6843	0.8738
XGBoost	0.8103	0.6791	0.8144	0.7406	0.8927

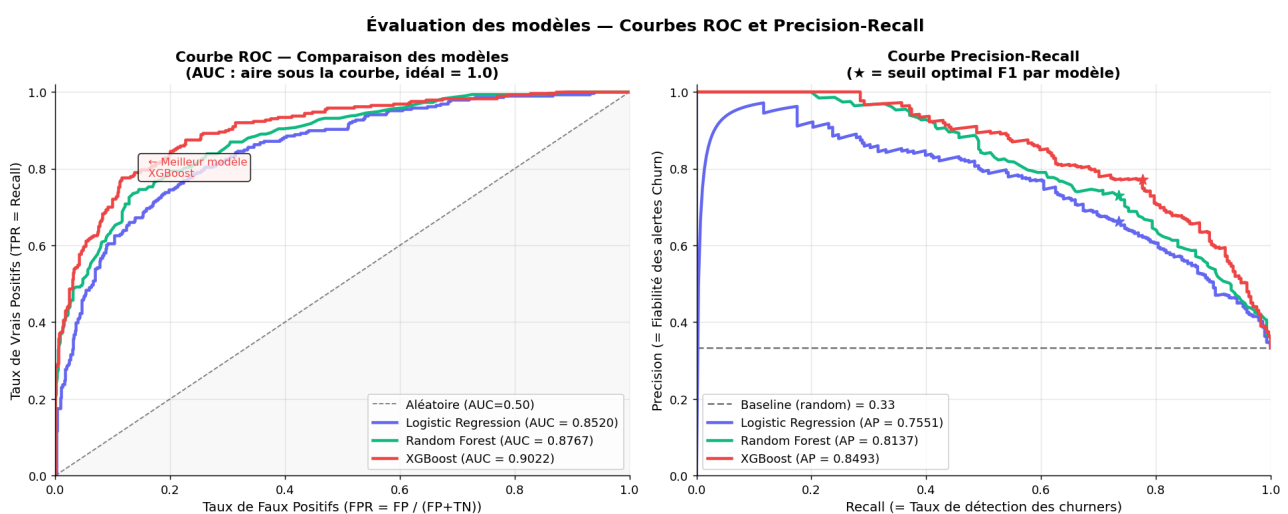


FIGURE 6 – Évaluation des modèles

4.5 Conclusion de la phase d'entraînement

Les résultats montrent que :

- La régression logistique constitue une baseline interprétable mais limitée.
- Le Random Forest améliore la robustesse et la performance globale.
- Le modèle XGBoost offre les meilleures performances globales, avec un bon compromis entre précision et capacité de détection.

Ainsi, **XGBoost est retenu comme modèle final pour la phase d'optimisation.**

5 Optimisation des hyperparamètres

Deux approches ont été utilisées pour optimiser les hyperparamètres du modèle XGBoost : GridSearchCV et Optuna.

5.0.1 GridSearchCV

GridSearchCV effectue une recherche exhaustive sur une grille fixe d'hyperparamètres avec validation croisée.

Configuration :

Paramètre	Valeur
Validation croisée	5-fold stratifiée
Métrique	F1-score
Nombre de combinaisons	144
Nombre total d'entraînements	720

Meilleurs hyperparamètres :

Hyperparamètre	Valeur
learning_rate	0.05
max_depth	7
min_child_weight	1
n_estimators	200
reg_lambda	5
subsample	1.0

Résultats :

Métrique	Score
Accuracy	0.8000
Precision	0.6518
Recall	0.8557
F1-score	0.7400
AUC-ROC	0.8919
CV F1	0.8651 ± 0.0063

5.0.2 Optuna (Optimisation bayésienne)

Optuna utilise une stratégie d'optimisation probabiliste (TPE) qui apprend progressivement les zones les plus prometteuses de l'espace des hyperparamètres.

Configuration :

Paramètre	Valeur
Nombre de trials	100
Métrique optimisée	AUC-ROC
Stratégie	TPE (Bayésienne)

Meilleurs hyperparamètres :

Hyperparamètre	Valeur
n_estimators	292
max_depth	4
learning_rate	0.0489
subsample	0.797
colsample_bytree	0.540
min_child_weight	11
reg_lambda	2.05
reg_alpha	0.51
gamma	4.34

Résultats :

Métrique	Score
Accuracy	0.7829
Precision	0.6211
Recall	0.8900
F1-score	0.7316
AUC-ROC	0.8945
CV AUC-ROC	0.9490

5.1 Comparaison des méthodes d'optimisation

Méthode	Accuracy	Precision	Recall	F1-score	AUC-ROC
GridSearchCV	0.8000	0.6518	0.8557	0.7400	0.8919
Optuna	0.7829	0.6211	0.8900	0.7316	0.8945

5.2 Analyse de l'importance des variables

L'analyse de l'importance des variables permet d'identifier les facteurs les plus influents dans la prédiction du churn. Les résultats obtenus à partir du modèle XGBoost sont présentés dans le tableau suivant.

Rang	Variable	Importance
1	SpendingCategory	0.1169
2	Frequency	0.0965
3	AccountAgeDays	0.0595
4	TotalTransactions	0.0532
5	RegSeason_automne	0.0442
6	IsNewClient	0.0411
7	ProductDiversity_Explorateur	0.0365
8	AvgDaysBetweenPurchases	0.0312
9	RegSeason_été	0.0291
10	Gender_Unknown	0.0278
11	RegSeason_hiver	0.0248
12	Gender_F	0.0211
13	SupportTicketsCount	0.0210
14	PreferredDayOfWeek	0.0201
15	WeekendPreference_Semaine	0.0180
16	MonetaryTotal	0.0176
17	RegMonth	0.0175
18	Region	0.0162
19	AvgBasketValue	0.0162
20	RegWeekday	0.0155

5.3 Analyse de l'overfitting du modèle

L'analyse des performances du modèle XGBoost optimisé par Optuna met en évidence un écart significatif entre les performances d'entraînement, de validation croisée et de test.

Métrique	Train	CV (mean)	Test
F1-Score	0.8826	0.8527	0.7435
AUC-ROC	0.9526	—	0.8945

Analyse des écarts :

- **Train - CV** : +0.0298 → écart faible, validation croisée globalement cohérente
- **CV - Test** : +0.1092 → baisse importante sur le jeu de test
- **Train - Test** : +0.1390 → indication claire de surapprentissage

La faible variance des scores en validation croisée ($\text{std} = 0.0084$) montre cependant que le modèle reste stable entre les folds, ce qui suggère que le problème ne provient pas de l'instabilité du modèle mais plutôt d'une légère sur-optimisation sur les données d'entraînement.

Conclusion : le modèle présente un surapprentissage modéré à sévère, avec une bonne stabilité en validation croisée mais une perte de généralisation sur le jeu de test.

5.3.1 Interprétation des résultats

Les résultats montrent que les variables les plus influentes sont principalement liées au comportement d'achat et à l'engagement client.

La variable **SpendingCategory** est la plus discriminante, ce qui indique que le niveau de dépenses est un facteur clé dans la probabilité de churn. Elle est suivie par la **fréquence d'achat**, ce qui confirme que les clients actifs sont moins susceptibles de quitter le service.

Les variables temporelles telles que **AccountAgeDays** et **TotalTransactions** jouent également un rôle important, ce qui montre que la fidélité et l'ancienneté du client sont des indicateurs forts de rétention.

Par ailleurs, certaines variables saisonnières (RegSeason) et comportementales (ProductDiversity, AvgDaysBetweenPurchases) apparaissent également dans le top des features, ce qui souligne l'importance des habitudes d'achat dans la prédiction du churn.

Enfin, des variables démographiques et contextuelles comme le genre, la région ou les tickets de support ont une influence plus modérée mais restent contributives au modèle.

5.4 Conclusion

Conclusion de l'optimisation

Optuna est retenu comme méthode finale d'optimisation grâce à :

- Une meilleure performance globale (AUC-ROC = 0.8945)
- Une meilleure capacité de détection des churners (Recall élevé)
- Une exploration plus efficace de l'espace des hyperparamètres

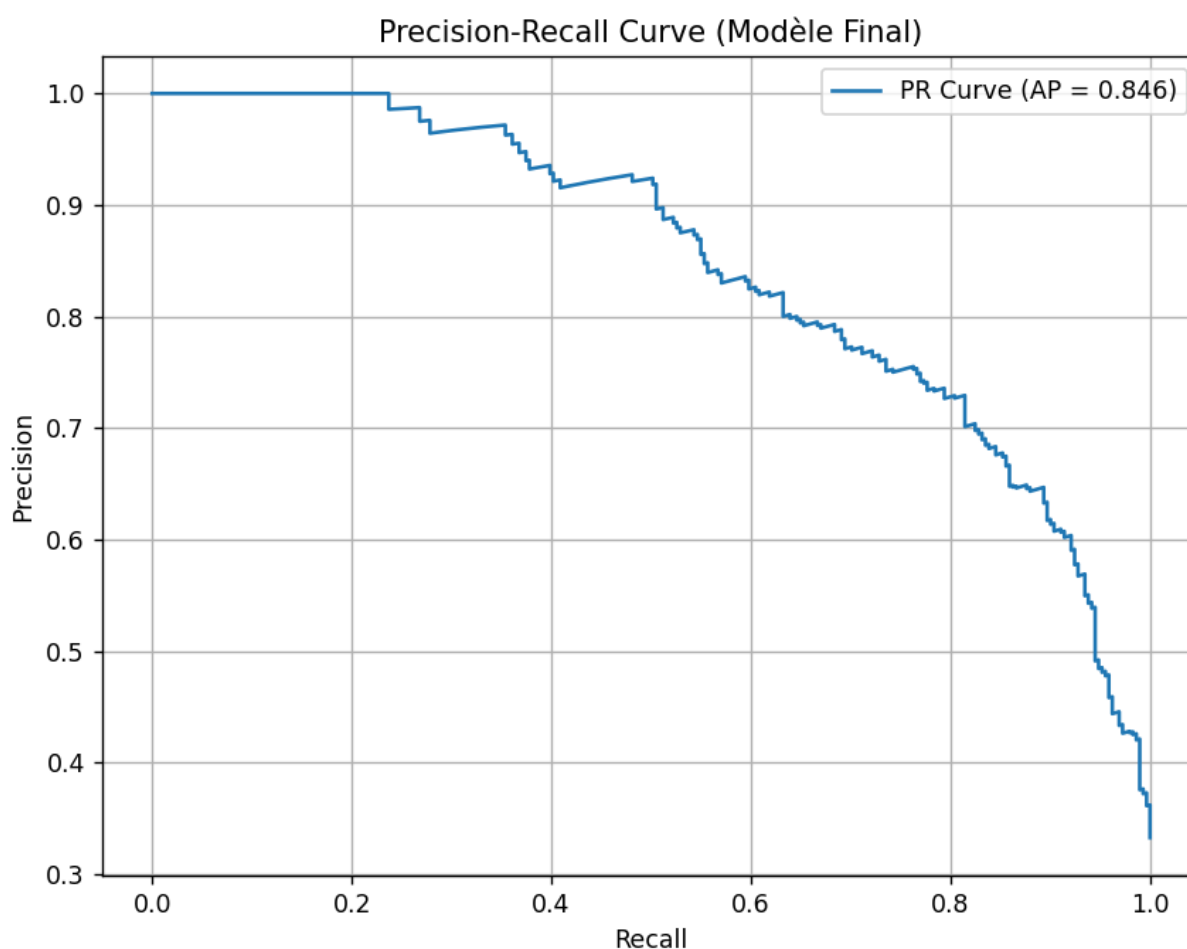


FIGURE 7 – Curve Precision-Recall

6 Déploiement — Application Web Flask

Le modèle de prédiction du churn est déployé via une application Flask structurée autour d'un pipeline complet de prétraitement, reproduisant fidèlement celui utilisé lors de l'entraînement. L'architecture repose sur deux niveaux :

- Une API REST (`predict.py`) pour les prédictions au format JSON.
- Une interface web (`app.py`) pour l'interaction avec l'utilisateur.

6.1 Architecture des endpoints

Méthode	Endpoint	Rôle
GET	<code>/health</code>	Vérifie que le modèle est chargé et que l'API fonctionne correctement
GET	<code>/model/info</code>	Retourne les informations du modèle (features, type, seuil de décision)
POST	<code>/predict</code>	Prédit le churn pour un client unique
POST	<code>/predict/batch</code>	Prédictions pour plusieurs clients et calcul du churn rate global

6.2 Pipeline de prétraitement en production

Le fichier `predict.py` reproduit exactement le pipeline d'entraînement :

Étape	Transformation	Description
1	Nettoyage	Suppression des colonnes inutiles et clipping des valeurs aberrantes
2	Parsing des dates	Création de <code>AccountAgeDays</code> , <code>RegMonth</code> , <code>RegSeason</code> et <code>IsNewClient</code>
3	Feature Engineering	<code>AvgBasketValue</code> , <code>CancellationRate</code> , <code>EngagementScore</code> , etc.
4	Imputation	Imputation médiane et mode via <code>imputers.pkl</code>
5	Encodage	Ordinal, Target et One-Hot Encoding
6	Multicolinéarité	Suppression des variables corrélées (<code>metadata.pkl</code>)
7	Scaling	Normalisation avec <code>RobustScaler</code>
8	Alignement	Ordre des features identique à celui du modèle <code>XGBoost</code>

6.3 Features les plus importantes du modèle

Le modèle `XGBoost` repose principalement sur des variables comportementales et transactionnelles :

Rang	Feature	Importance
1	<code>SpendingCategory</code>	0.1169
2	<code>Frequency</code>	0.0965
3	<code>AccountAgeDays</code>	0.0595
4	<code>TotalTransactions</code>	0.0532
5	<code>RegSeason (automne)</code>	0.0442
6	<code>IsNewClient</code>	0.0411
7	<code>ProductDiversity</code>	0.0365
8	<code>AvgDaysBetweenPurchases</code>	0.0312

Ces résultats montrent que le churn est principalement influencé par :

- la fréquence d'achat (`Frequency`),
- le niveau de dépense (`SpendingCategory`),
- l'ancienneté du client (`AccountAgeDays`),
- et le niveau d'engagement (transactions et diversité produit).

6.4 Exemple d'utilisation de l'API

```

1 POST /predict
2 Content-Type: application/json
3
4 {
5     "Frequency": 2,
6     "MonetaryTotal": 150.0,
7     "MonetaryAvg": 60.0,
8     "MonetaryStd": 20.0,
9     "TotalTransactions": 3,
10    "NegativeQuantityCount": 1,
11    "SatisfactionScore": 2,
12    "SupportTicketsCount": 8,
13    "AvgDaysBetweenPurchases": 40,
14    "UniqueProducts": 2,
15    "SpendingCategory": "Low",
16    "BasketSizeCategory": "Petit",
17    "Region": "UK",
18    "RegistrationDate": "15/03/2016"
19 }

```

Listing 1 – Requête API — client à risque

Réponse JSON de l'API

```

{
  "churn_prediction": 1,
  "churn_prediction_optimal": 1,
  "churn_label": "Oui - Client à risque",
  "churn_probability": 0.999,
  "risk_level": "Critique"
}

```

FIGURE 8 – Réponse de l'API Flask pour un client à risque

6.5 Règle de décision (Risk Engine)

Le système de décision repose sur des seuils de probabilité :

Probabilité	Niveau de risque
< 0.25	Faible
0.25 – 0.50	Moyen
0.50 – 0.75	Élevé
> 0.75	Critique

6.6 Tests manuels des prédictions

Afin d'évaluer la robustesse du modèle en situation réelle, plusieurs profils clients représentatifs ont été construits et soumis au modèle final (XGBoost optimisé par Optuna).

Deux seuils de décision sont analysés :

- Seuil standard : 0.5
- Seuil optimisé : 0.483 (issu de l'optimisation Precision-Recall)

6.6.1 Client VIP fidèle

Ce client représente un utilisateur très engagé avec une forte fidélité et une activité élevée.

- Probabilité de churn : 8.66%
- Niveau de risque : Faible
- Prédiction :
 - Seuil 0.5 : NON
 - Seuil optimal : NON

Interprétation : le modèle identifie correctement ce client comme non risqué grâce à une forte fréquence d'achat, une ancienneté élevée et un panier moyen élevé.

6.6.2 Client à risque de churn

Ce profil correspond à un client fortement désengagé et proche de la perte.

- Probabilité de churn : 76.85%
- Niveau de risque : Critique
- Prédiction :
 - Seuil 0.5 : OUI
 - Seuil optimal : OUI

Interprétation : le modèle détecte correctement un client à haut risque, caractérisé par une faible fréquence d'achat, un taux de retour élevé et une satisfaction faible.

6.6.3 Client moyen occasionnel

Ce client présente un comportement intermédiaire entre fidélité et désengagement.

- Probabilité de churn : 40.32%
- Niveau de risque : Moyen
- Prédiction :
 - Seuil 0.5 : NON
 - Seuil optimal : OUI

Interprétation : ce cas illustre l'intérêt du seuil optimisé, qui permet de mieux capter les clients à risque modéré non détectés par le seuil classique.

6.6.4 Conclusion des tests manuels

Conclusion

Les tests manuels confirment que :

- Le modèle distingue correctement les profils extrêmes (VIP vs risque élevé)
- Le seuil optimisé améliore la détection des cas intermédiaires
- Les probabilités fournissent une mesure fiable du risque de churn

6.7 Conclusion du déploiement

L'application Flask permet un déploiement complet du modèle XGBoost avec :

- un pipeline de prétraitement strictement identique à celui de l'entraînement,
- une API REST prête pour intégration système,
- une interprétation métier via niveaux de risque,
- une extensibilité vers un tableau de bord et du monitoring.

7 Structure du Projet

Arborescence du projet

```

projet_ml_retail/
data/
raw/          ← CSV original (ne jamais modifier)
processed/    ← Données nettoyées intermédiaires
train_test/   ← X_train, X_test, y_train, y_test (générés)
notebooks/    ← Exploration Jupyter (EDA visuelle)
src/          ← Scripts de production Python
utils.py      ← Chargement + EDA + correction conflit Git
preprocessing.py ← Pipeline 9 étapes
train_model.py ← 3 modèles + KMeans + GridSearch + Optuna
predict.py    ← API REST JSON Flask
app/          ← Interface web Flask
app.py        ← Routes web (formulaire + dashboard)
templates/    ← index.html, result.html, dashboard.html
models/       ← Modèles .pkl sauvegardés
best_model.pkl ← Modèle final (XGBoost Optuna)
scaler.pkl / imputers.pkl / encoders.pkl
pca.pkl / pca_metadata.pkl
kmeans.pkl / optuna_study.pkl
reports/      ← Visualisations et rapports
main.py       ← Point d'entrée unique
requirements.txt ← Dépendances
README.md

```

Dépendances principales (requirements.txt) :

Librairie	Version	Rôle
pandas	≥ 2.0.0	Manipulation et analyse des données
numpy	≥ 1.24.0	Calcul scientifique et opérations matricielles
scikit-learn	≥ 1.3.0	Machine Learning : preprocessing, modèles et métriques
xgboost	≥ 1.7.0	Modèle principal de classification (gradient boosting)
optuna	≥ 4.0.0	Optimisation bayésienne des hyperparamètres
imbalanced-learn	≥ 0.11.0	Gestion du déséquilibre des classes (SMOTE)
flask	≥ 3.0.0	Déploiement API REST et interface web
joblib	≥ 1.3.0	Sauvegarde et chargement des modèles sérialisés (.pkl)
matplotlib	≥ 3.8.0	Visualisation des données et des performances du modèle
category-encoders	≥ 2.6.0	Encodage des variables catégorielles (Target Encoding, etc.)

8 Conclusion

Ce projet a permis de construire un système complet de prédiction du churn client, depuis l'exploration des données brutes jusqu'au déploiement d'une application web fonctionnelle. Les principaux apprentissages sont :

Bilan des apprentissages clés

- ✓ **Data Leakage** La découverte et suppression des colonnes **ChurnRiskCategory** et **CustomerType** illustre l'importance de comprendre les données avant de modéliser. Ces colonnes auraient généré des performances artificiellement parfaites.
- ✓ **Feature Engineering** Les 8 nouvelles features créées (**TenureRatio**, **SpendingVolatility...**) apparaissent dans le top 10 des features importantes, validant l'apport de la connaissance métier dans le ML.
- ✓ **Préprocessing rigoureux** La règle fit-sur-train/transform-sur-test a été respectée à chaque étape (imputation, encodage, normalisation, ACP) pour éviter tout data leakage.
- ✓ **GridSearch + Optuna** La stratégie en deux étapes (GridSearch pour explorer, Optuna pour affiner) illustre comment combiner deux outils complémentaires d'optimisation.
- ✓ **Pipeline industrialisé** L'architecture en modules indépendants (**utils**, **preprocessing**, **train_model**, **predict**, **app**) permet de maintenir, tester et faire évoluer chaque composant séparément.

Résultats Finaux du Modèle XGBoost (Optuna)

F1-Score : 0.7316 | AUC-ROC : 0.8945 | Recall : 0.8900 | Precision : 0.6211

Sur 291 churners dans le jeu de test : **259 correctement détectés** (32 manqués)

Feature la plus importante : **SpendingCategory** (importance = 0.1169), suivie de **Frequency** et **AccountAgeDays**

Pipeline complet reproductible : `python main.py` — de la donnée brute au modèle en ~2 minutes

Perspectives d'amélioration des performances du modèle :

- Exploration de modèles plus robustes au bruit et au surapprentissage (ex : LightGBM ou CatBoost) afin de comparer les performances avec XGBoost
- Calibration des probabilités (Platt Scaling ou Isotonic Regression) pour améliorer l'interprétation des scores de churn
- Mise en place d'une sélection automatique de features plus avancée (ex : SHAP-based feature selection) pour réduire encore la variance du modèle
- Étude de modèles d'ensemble (Stacking ou Blending) combinant plusieurs algorithmes afin d'améliorer la généralisation
- Optimisation métier du seuil de décision en fonction du coût réel des faux négatifs (clients churn non détectés)
- Renforcement du suivi en production via monitoring du data drift et dégradation des performances dans le temps